PropOS 角色体系扩展计划

> 版本: v1.0
>
> 日期: 2026-04-13
>
> 状态: 已批准
>
> 依据: PRD v1.7 / RBAC_MATRIX v1.0 / ARCH v1.2 / 行业参照分析（Yardi、MRI、Buildium）

一、变更背景

1.1 行业差距分析

主流物业 SaaS（Yardi、MRI、Buildium）典型角色层级为 8–12 个，涵盖「系统管理、资产管理、租务、财务、工程维修、巡检楼管、客户服务、只读审计、供应商门户、租户门户」。

PropOS 当前定义 6 个角色（含 `sub_landlord`），存在 3 个实质性缺口：

缺口	影响	优先级
`frontline_staff` 过于宽泛	维修工、楼管巡检、客服混为一体，权限粗粒度	高
缺少只读利益相关者角色	投资人/审计/只读决策层无法安全访问报表	高
`operations_manager` 权限过载	Phase 2+ 业态细分时缺少子角色承接	中（Phase 2 解决）

1.2 已发现的技术债

后端 Dart 实现与文档之间存在系统性命名偏移，需一并修正：

角色	文档标识符	后端实际 API string	偏差说明
超级管理员	`super_admin`	`admin`	文档使用 `super_admin`，后端缩写为 `admin`
租务专员	`leasing_specialist`	`lease_specialist`	文档多一个字母 `ing`
前线员工	`frontline_staff`	*(不存在)*	后端已演进为 `maintenance_staff`
只读角色	*(文档未收录)*	`read_only`	后端已实现，文档从未定义

裁决原则：文档命名胜出（标准化），后端迁移对齐。

二、目标角色集（8 个）

2.1 角色定义表

#	API 标识符（snake_case）	Dart 枚举（camelCase）	中文名	定位	数据范围	变更类型
1	`super_admin`	`superAdmin`	超级管理员	全系统控制	全部数据	重命名：后端 `admin` → `super_admin`
2	`operations_manager`	`operationsManager`	运营管理层	业务决策 + 审批	管辖范围内数据	✅ 无变更
3	`leasing_specialist`	`leasingSpecialist`	租务专员	合同 + 租客日常操作	管辖范围内数据	重命名：后端 `lease_specialist` → `leasing_specialist`
4	`finance_staff`	`financeStaff`	财务人员	财务收支 + 核销	全部财务数据	✅ 无变更
5	`maintenance_staff`	`maintenanceStaff`	维修技工	工单接派 + 水电抄表	管辖范围内工单	重命名：文档 `frontline_staff` → `maintenance_staff`（对齐后端）
6	`property_inspector`	`propertyInspector`	楼管巡检员	巡检登记 + 资产查看	管辖楼栋/楼层	全新
7	`report_viewer`	`reportViewer`	只读观察者	报表查看 + 投资人/审计	全部只读数据	重命名：后端 `read_only` → `report_viewer`
8	`sub_landlord`	`subLandlord`	二房东	自身子租赁填报	仅 `bound_contract_id`	✅ 无变更

2.2 与行业标准对照

```
Yardi/MRI 标准        PropOS 映射
———————————————————————————————————————
System Admin            →     super_admin
Portfolio Manager       →     operations_manager（Phase 2 可拆分业态子角色）
Property Manager        →     operations_manager（数据范围限定）
Leasing Agent           →     leasing_specialist
Accounting              →     finance_staff
Maintenance Supervisor  →     operations_manager（派单审批）
Technician              →     maintenance_staff ← 新拆分
Property Inspector      →     property_inspector ← 新增
Auditor / Viewer        →     report_viewer ← 新增
Vendor Portal           →     Phase 2
Tenant Portal           →     Phase 2
Sub-Landlord Portal     →     sub_landlord
```

三、权限矩阵（8 角色 × 25 权限）

✅ = 拥有　❌ = 无权限

权限字符串	super_admin	operations_manager	leasing_specialist	finance_staff	maintenance_staff	property_inspector	report_viewer	sub_landlord
`org.read`	✅	✅	✅	✅	❌	✅	✅	❌
`org.manage`	✅	✅	❌	❌	❌	❌	❌	❌
`assets.read`	✅	✅	✅	✅	❌	✅	✅	❌
`assets.write`	✅	✅	❌	❌	❌	❌	❌	❌
`contracts.read`	✅	✅	✅	✅	❌	✅[1]	✅[2]	❌
`contracts.write`	✅	✅	✅	❌	❌	❌	❌	❌
`deposit.read`	✅	✅	✅	✅	❌	❌	✅	❌
`deposit.write`	✅	✅	❌	✅	❌	❌	❌	❌
`finance.read`	✅	✅	✅[3]	✅	❌	❌	✅	❌
`finance.write`	✅	❌	❌	✅	❌	❌	❌	❌
`kpi.view`	✅	✅	✅	✅	✅	✅	✅	❌
`kpi.manage`	✅	✅	❌	❌	❌	❌	❌	❌
`kpi.appeal`	✅	✅	✅	✅	✅	✅	❌	❌
`meterReading.write`	✅	✅	✅	✅	✅	✅	❌	❌
`turnoverReview.approve`	✅	✅	❌	✅	❌	❌	❌	❌
`workorders.read`	✅	✅	✅	❌	✅	✅	❌	❌
`workorders.write`	✅	✅	❌	❌	✅	❌	❌	❌
`sublease.read`	✅	✅	✅	✅	❌	❌	✅	❌
`sublease.write`	✅	✅	✅	❌	❌	❌	❌	❌

`sublease.portal`	❌	❌	❌	❌	❌	❌	❌	✅
`alerts.read`	✅	✅	✅	✅	❌	❌	✅	❌
`alerts.write`	✅	✅	❌	❌	❌	❌	❌	❌
`ops.read`	✅	✅	❌	❌	❌	❌	❌	❌
`ops.write`	✅	❌	❌	❌	❌	❌	❌	❌
`import.execute`	✅	✅	✅[4]	✅[4]	❌	❌	❌	❌

注释：

1. `property_inspector` 的 `contracts.read` 限制为仅查看租客基本信息（姓名、房号），不可查看财务金额字段
2. `report_viewer` 的 `contracts.read` 限制为不可查看租客敏感信息（证件号、手机号等 PII）
3. `leasing_specialist` 的 `finance.read` 限制为仅查看与自身管辖合同关联的账单，不可查看全局 NOI 报表
4. `leasing_specialist` 和 `finance_staff` 仅可执行各自领域的导入

3.1 新增角色权限设计理由

`maintenance_staff`（维修技工）：

- 来源：原 `frontline_staff` 精细化拆分
- 核心操作：工单接单/完工/拍照 + 水电抄表录入
- 排除项：无合同/资产/财务查看权，防止越权
- 行业对标：Yardi Technician 角色

`property_inspector`（楼管巡检员）：

- 来源：行业需求，物管公司标配巡检岗位
- 核心操作：资产信息查看 + 水电抄表 + 工单查看（不可创建/处理）+ 租客基本信息查看
- 排除项：无财务/押金权限，无工单写入权限（发现问题通知运营层处理）
- 行业对标：Property Inspector / Building Attendant

`report_viewer`（只读观察者）：

- 来源：投资人/股东/外部审计/集团管理层 需安全查看运营数据
- 核心操作：NOI 报表 + 出租率 + WALE + KPI 概览 + 资产/合同摘要
- 排除项：零 write 权限，无 PII 访问，无操作审计日志
- 安全考量：最小权限原则，write 操作完全隔离
- 行业对标：Yardi Investor Portal / MRI Read-Only User

四、实施计划

Phase 0：设计基线 ✅（本文档即为产出）

Phase 1：文档更新（文档先行，阻塞后续阶段）

步骤	文件	更新内容	预估复杂度
1.1	`docs/backend/RBAC_MATRIX.md`	完整重写：8角色定义表、权限矩阵（8列）、全量端点映射	高
1.2	`docs/PRD.md`	Section 二用户角色表：8行角色、frontline→maintenance 说明	中
1.3	`docs/ARCH.md`	JWT claims 表、Admin 访问矩阵、Row-Level Isolation 段落	中
1.4	`docs/backend/data_model.md`	`user_role` SQL enum 定义更新至 8 值	低
1.5	`docs/backend/SEED_DATA_SPEC.md`	新增 3 个 seed 用户：维修工/巡检员/只读观察者	低
1.6	`docs/backend/NOTIFICATION_PUSH.md`	通知收件人角色更新	低
1.7	`docs/backend/API_INVENTORY_v1.7.md`	角色缩写表扩展至 8 个；端点级角色列刷新	中
1.8	`docs/frontend/RBAC_PROTOTYPE_PLAN.md`	角色-人物映射扩展至 8 人	低
1.9	`docs/frontend/PAGE_SPEC_v1.8.md`	Role-based UI 规格新增 3 角色视图	中
1.10	`docs/frontend/plan/FINANCE_ROLE_ADAPTIVE_DESIGN.md`	新增 3 个角色的财务页视图规格	中

Phase 2：后端代码更新（依赖 Phase 1）

步骤	文件	更新内容
2.1	`backend/lib/core/request_context.dart`	`UserRole` 枚举重命名 3 个 + 新增 1 个 + `fromString()` / `get value` 映射
2.2	`backend/lib/core/middleware/rbac_middleware.dart`	权限矩阵 Map 重映射 + 新增 `propertyInspector` 权限集
2.3	`backend/migrations/YYYYMMDD_role_expansion.sql`	PostgreSQL enum 安全迁移（create → cast → drop 三步式）

Phase 3：前端原型更新（React，依赖 Phase 1，可与 Phase 2 并行）

步骤	文件	更新内容
3.1	`frontend/src/app/auth/types.ts`	`Role` union type → 8 元素
3.2	`frontend/src/app/auth/permissions.ts`	`ROLE_PERMISSIONS` 新增 3 角色 + `MOCK_USERS` 新增 3 人 + 更新特殊检查
3.3	`frontend/src/app/auth/AuthContext.tsx`	Role switcher 列表 8 角色
3.4	`frontend/src/app/routes.tsx`	`TAB_PERMISSIONS` / `ROUTE_RULES` 更新
3.5	`frontend/src/app/pages/Finance.tsx`	新增 3 角色分支渲染
3.6	`frontend/src/app/pages/Profile.tsx`	Role switcher 显示 8 角色
3.7	`frontend/src/app/pages/Home.tsx`	`canAccessGlobalNOI` 扩展

Phase 4：Admin / App 类型同步（可与 Phase 3 并行）

步骤	文件	更新内容
4.1	`admin/src/types/api.ts`	新增 `UserRole` type（8 角色 union）
4.2	`admin/src/stores/auth.ts`	`UserProfile.role: string` → `UserRole`
4.3	`admin/src/router/index.ts`	`beforeEach` 守卫加入角色校验
4.4	`app/src/types/api.ts`	新增 `UserRole` type
4.5	`app/src/api/modules/auth.ts`	`UserProfile.role: string` → `UserRole`
4.6	`app/src/stores/auth.ts`	角色 computed 类型更新

五、数据库迁移策略

5.1 PostgreSQL 枚举安全迁移

PostgreSQL 不支持直接 `ALTER TYPE ... RENAME VALUE`，需采用三步安全迁移：

```
BEGIN;

-- Step 1: 创建新枚举类型
CREATE TYPE user_role_new AS ENUM (
  'super_admin',
  'operations_manager',
  'leasing_specialist',
```

```
  'finance_staff',
  'maintenance_staff',
  'property_inspector',
  'report_viewer',
  'sub_landlord'
);

-- Step 2: 迁移列（含旧值映射）
ALTER TABLE users
  ALTER COLUMN role TYPE user_role_new
  USING (
    CASE role::text
      WHEN 'admin' THEN 'super_admin'
      WHEN 'lease_specialist' THEN 'leasing_specialist'
      WHEN 'frontline_staff' THEN 'maintenance_staff'
      WHEN 'read_only' THEN 'report_viewer'
      ELSE role::text
    END
  )::user_role_new;

-- Step 3: 替换旧类型
DROP TYPE user_role;
ALTER TYPE user_role_new RENAME TO user_role;

COMMIT;
```

5.2 关联表影响

需检查所有引用 `user_role` 枚举的表/列：

- `users.role` （主表）
- 审计日志中可能的角色记录字段
- 任何 `CHECK` 约束或默认值

5.3 回滚方案

```
-- 回滚脚本（保留在迁移文件的 DOWN 部分）
-- 反向映射：将新值转回旧值
-- 注意：新增的 property_inspector 用户需先手动处理
```

六、受影响文件清单

文档（10 个）

1. `docs/backend/RBAC_MATRIX.md` （主源）
2. `docs/PRD.md`
3. `docs/ARCH.md`
4. `docs/backend/data_model.md`

5. `docs/backend/SEED_DATA_SPEC.md`
6. `docs/backend/NOTIFICATION_PUSH.md`
7. `docs/backend/API_INVENTORY_v1.7.md`
8. `docs/frontend/RBAC_PROTOTYPE_PLAN.md`
9. `docs/frontend/PAGE_SPEC_v1.8.md`
10. `docs/frontend/plan/FINANCE_ROLE_ADAPTIVE_DESIGN.md`

后端代码（3 + 1 新建）

1. `backend/lib/core/request_context.dart`
2. `backend/lib/core/middleware/rbac_middleware.dart`
3. `backend/migrations/20260413_role_expansion.sql` （新建）

前端原型 React（7 个）

1. `frontend/src/app/auth/types.ts`
2. `frontend/src/app/auth/permissions.ts`
3. `frontend/src/app/auth/AuthContext.tsx`
4. `frontend/src/app/routes.tsx`
5. `frontend/src/app/pages/Finance.tsx`
6. `frontend/src/app/pages/Profile.tsx`
7. `frontend/src/app/pages/Home.tsx`

Admin Vue 3（3 个）

1. `admin/src/types/api.ts`
2. `admin/src/stores/auth.ts`
3. `admin/src/router/index.ts`

App Flutter（3 个）

1. `flutter_app/lib/core/api/api_client.dart`
2. `flutter_app/lib/features/auth/data/repositories/auth_repository_impl.dart`
3. `flutter_app/lib/features/auth/presentation/bloc/auth_cubit.dart`

总计：**26 个文件**（10 文档 + 4 后端 + 7 前端原型 + 3 Admin + 3 App）

七、角色缩写对照（更新后）

缩写	角色
SA	`super_admin`
OM	`operations_manager`
LS	`leasing_specialist`
FS	`finance_staff`
MS	`maintenance_staff`
PI	`property_inspector`
RV	`report_viewer`
SL	`sub_landlord`

八、验证清单

- [ ] `RBAC_MATRIX.md` 角色列数为 8，无 `frontline_staff` 、无旧命名 `admin` / `lease_specialist` / `read_only`
- [ ] 后端 `UserRole.fromString()` 所有 API 标识符与 `RBAC_MATRIX.md` 完全一致
- [ ] 数据库迁移脚本在本地 PostgreSQL 可 dry run（事务回滚验证，无数据丢失）
- [ ] 前端原型 Profile 页 role switcher 显示 8 个角色
- [ ] `can("finance.read")` + `canAccessGlobalNOI(role)` 对 `report_viewer` 均返回 `true`
- [ ] Admin / App TypeScript 严格模式编译无 `role` 类型错误
- [ ] 所有 10 个文档中的角色标识符全局搜索无遗留旧名

九、风险与缓解

风险	影响	缓解措施
DB 枚举迁移失败	阻塞部署	事务包裹 + 预发环境验证 + 回滚脚本
前端硬编码角色名未全量替换	UI 权限穿透	全局搜索 `frontline_staff` 、 `admin` （非前缀匹配）验证零命中
`frontline_staff` 下辖用户需按人手动分流	迁移后权限不匹配	默认迁移为 `maintenance_staff` ；DBA 人工确认后执行
`report_viewer` 误获 PII	合规风险	Repository 层继续对证件号/手机号脱敏， `contracts.read` 行级过滤验证